Answering a busy group chat without a backlog

A design for an agent that works on many chat requests at once, still gives every reader a legible stream of answers, and survives people changing their minds while it is already working.

Jay Talwar Written before the code 18 October 2026

Before you start

This document assumes no background. Every technical term is defined where it first appears, and there is a glossary at the end. If you hit a word you don't recognise, it is defined somewhere above — or it shouldn't be there.

Here is the whole world the design lives in:

- An **agent** is a program that answers questions using a language model. To answer "am I overweight energy?" it has to *look things up* — fetch your holdings, fetch prices. Those lookups are called **tool calls**, and they are the slow, expensive part. A single question might take twenty seconds and cost a few cents.
- Cost is measured in **tokens** — roughly, pieces of words. Everything the agent reads and writes is billed by the token. When this document says work was "wasted", it means tokens we paid for and threw away.
- The agent sits in a **group chat** with several people in it. They talk at the same time, interrupt, change their minds, and ask follow-ups.

That is it. Everything else is built up from here.

§ 01

The problem, concretely

Castle's agent sits in a group chat. When the chat gets busy, several people ask it several things at once. Today it answers them one at a time, so a queue forms. The brief asks for a system that works on many requests **in parallel** — several at the same time, rather than finishing one before starting the next — still gives each chat a **clear, ordered stream of answers**, and handles people **changing or adding to a request while it is running**.

Here is the situation the system has to survive. Times are seconds from the start of the burst.

t=0	**Ana**	What's my exposure to a Taiwan escalation?	NEW · SLOW
t=2	**Ben**	Is NVDA still my biggest position?	NEW · FAST
t=4	**Ana**	Actually make that Taiwan *or* South China Sea.	CHANGES A RUNNING REQUEST
t=5	**Cara**	What's my exposure to Taiwan?	SAME QUESTION ANA ASKED
t=9	**Ben**	And the second biggest?	NEEDS AN EARLIER ANSWER
t=11	**Cara**	never mind the Taiwan one	CANCELS

Six messages, three pieces of work. Two of the three change shape after they have already started. This transcript is the specification — and scenario 1 of the test suite.

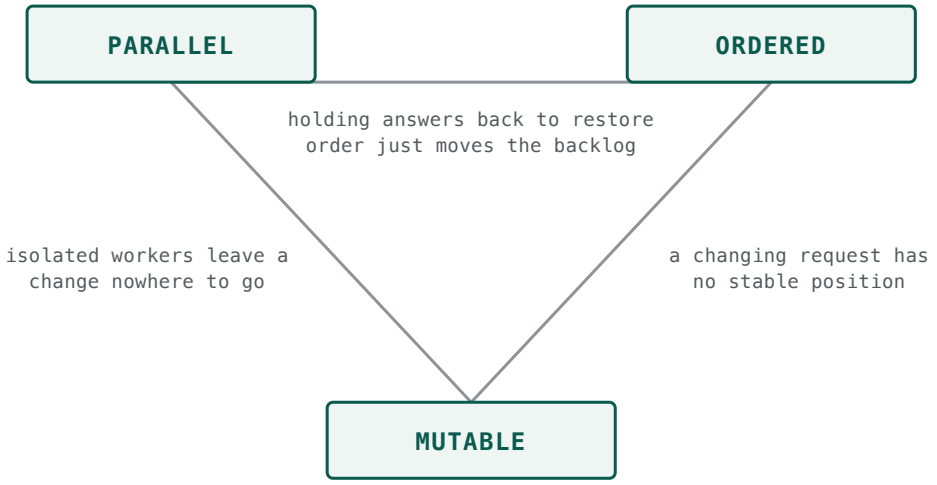
> Throughout this document, a **`request`** means one piece of work the agent owes somebody an answer for. A **`message`** is one line somebody typed. The two are not the same thing, and § 04 argues that confusing them is the central mistake available here.

A system that treats each message as an independent job gets this transcript wrong in at least four ways. It answers the Taiwan question twice — Ana and Cara asked the same thing. It answers Ana's original question, which she already replaced. It answers "the second biggest?" before anyone knows what the first one was. And it keeps paying for work Cara cancelled.

§ 02

Why the three requirements fight each other

The three requirements are not independently satisfiable. Every *pair* of them is in tension, which is why this is a design problem rather than a to-do list.



Each edge is a real conflict, not a tradeoff dial. The way through is to notice that one of the three corners is not what it appears to be.

Parallel × Ordered

Work finishes out of order — a question that takes one second finishes before one that takes thirty, whoever asked first. You could restore arrival order by holding finished answers back until their predecessors are done. But then a fast question asked third waits on a slow question asked first, and you have moved the backlog from *computing* answers to *delivering* them. The user sees no improvement.

> **Head-of-line blocking** — when the first item in a queue is slow, everything behind it waits, however fast those items are individually. It is the specific failure the brief describes as "answering them one at a time creates a backlog". Named after a queue of people at a door: one person fumbling for their ticket blocks everyone behind them.

Parallel × Mutable

A change — *"actually make that…"* — targets **one specific computation that is currently running**. The obvious way to get parallelism is to give each request its own isolated worker that shares nothing with the others. But then the change has nowhere to go: whoever receives the new message has no handle on the worker that needs to hear it, and no shared context in which to work out what "that" refers to.

Ordered × Mutable

"Ordered" presumes a stable sequence of things to order. If request #2 becomes a materially different request at t=30s — or is cancelled, or turns out to be the same question as request #4 — the sequence is not stable. Ordering by the arrival time of the *original* message is ordering by a fact that is no longer true.

§ 03

First reframe: "ordered" is about legibility, not sequence

Nobody in a group chat actually wants strict arrival order.

> **FIFO** — "first in, first out", the rule that things come out in the order they went in. A supermarket queue is FIFO. It is the obvious reading of "ordered", and it is the one that costs head-of-line blocking.

Pull on "a clear, ordered stream of answers" and ask what a reader actually needs, and it comes apart into four separate properties.

PROPERTY	WHAT IT MEANS FOR THE READER	NEEDS ORDERING?
Attribution	For every answer, I can tell which question it answers.	No
Completeness	I can tell what is still outstanding. Nothing silently vanished.	No
Non-contradiction	I never see two live answers that disagree. A replaced answer is visibly replaced.	No
Causal consistency	If answer B depends on answer A, A arrives first.	**Yes** — and only between dependent pairs

Three of the four need no ordering at all. FIFO delivery buys all four with one blunt instrument and charges head-of-line blocking for it.

Implemented directly, each property is cheap, and none of them blocks:

- **Attribution** → quote the question back: `re: "exposure to Taiwan" — …`
- **Completeness** → acknowledge each request when it arrives, and keep the outstanding set visible

- **Non-contradiction** → say “this replaces my earlier answer” when it does, and never ship an answer belonging to an out-of-date version of a request (§ 07)
- **Causal consistency** → record which request depends on which, and serialize *only* across that link

> THE DESIGN THESIS
>
> **Serialize on dependency, not on arrival.**

Serialize here means “force to happen one after another”. The claim is that the only thing that genuinely has to wait is an answer that needs another answer first — and nothing else.

Everything downstream follows from that sentence. It turns ordering from a structural property of the system into a **policy** that can be swapped per chat — which in turn makes it something we can *measure* rather than assert (§ 09, § 10).

§ 04

Second reframe: messages are not requests

The brief’s phrasing — “several questions at once” — invites you to treat each message as a job. In a real group chat that is wrong. Look at the transcript again: six messages, three requests.

So the right question on the way in is not *“what does this message ask for?”* but **“how does this message relate to the work already in flight?”** That distinction is load-bearing. It is what lets “And the second biggest?” create the dependency link § 03 needs, and what lets Cara’s repeat of Ana’s question avoid paying twice for one answer.

The five relationships we handle

RELATIONSHIP	MEANING	IN THE TRANSCRIPT
NEW	An independent request.	Ana t=0, Ben t=2
AMEND	Changes an existing request. Carries one flag, `preserves_premise`.	Ana t=4
CANCEL	Stop work; no answer is owed.	Cara t=11
DUPLICATE	Someone already asked this; one answer serves both.	Cara t=5

RELATIONSHIP	MEANING	IN THE TRANSCRIPT
REFERENCE	A new request that needs a previous request's *answer* first.	Ben t=9

The `preserves_premise` flag is doing real work. "Also include South China Sea" *widens* the question — the research already done on Taiwan is still valid, so we keep it. "Actually, not Taiwan, I meant inflation" *replaces* the premise — that research is now worthless and must be thrown away. Same relationship, one boolean, very different cost. Folding *widen*, *narrow* and *replace* into one class with one flag removes three categories the classifier would otherwise have to tell apart, and loses no behaviour.

Classifier errors must be asymmetric by design

> **`Classifier`** — a model whose job is to put a thing into one of a fixed set of categories. Here: given a message and the live request set, pick one of the five relationships above. **`Confidence`** is the classifier's own estimate of how likely it is to be right, from 0 to 1.

The classifier is a small language model and it will sometimes be wrong. What matters is making sure it is wrong in the **recoverable** direction.

- A `NEW` misclassified as `AMEND` **destroys a request.** The original question is overwritten, never answered, and nobody finds out.
- An `AMEND` misclassified as `NEW` **produces a redundant answer.** Noisy, but every question still gets answered and the reader can see what happened.

The second failure is recoverable and the first is not. So the classifier is **biased toward `NEW`**: each relationship has a confidence threshold it must clear, the destructive ones (`CANCEL` , and `AMEND` that replaces the premise) have a higher bar than the additive ones, and anything below its threshold falls back to `NEW` .

Because the classifier is the highest-risk component in the system, every decision it makes — proposed relationship, confidence, target, and whether a fallback was applied — is logged and printable. It is the one part a reviewer should be able to audit directly.

§ 05

Architecture

Four components. Only one of them holds state shared between requests, and it is deliberately the cheap one.

CHAT MESSAGES

append-only log on disk — replayable

every inbound message

TRIAGE

claude-haiku-4-5

Classifies each message against the live request set (§ 04) and emits one command. Runs on everything that arrives, so it must be cheap.

NEW / AMEND / CANCEL / DUPLICATE / REFERENCE

SEQUENCER — ONE PER CHAT

the only shared state · one event at a time · no model calls

- The register of live requests
- Which request depends on which
- **Admission** — what is allowed to start
- **Delivery** — what is allowed to ship

Pure bookkeeping.

start (request, version) → ← results, version-checked

WORKER POOL — BOUNDED

claude-opus-5

One isolated worker per request-version. Each has an inbox of pending changes that it reads only at safe moments.

answers

DELIVERY

Owns every reader-facing property from § 03: attribution, acknowledgements, replacement notices.

The shared-state component is the cheap one. Haiku runs once per inbound message; the Sequencer does no model calls at all; Opus runs only inside isolated, replicated workers.

`Worker` — one running job answering one request. **`Pool`** — the fixed set of workers that may run at once, say three. A request waits for a free **`slot`** before it starts. Bounding the pool is what stops a busy chat from launching forty expensive jobs at once.

Why not either sketch in the brief

Sketch 1 — a router plus parallel runtimes. A small model reads each message and sends it to its own isolated runtime. Good isolation, genuinely parallel. But the router is *stateless with respect to the live request set* — it does not know what is currently running. So it structurally cannot classify relationships: `AMEND`, `DUPLICATE` and `REFERENCE` are unrepresentable, because each of them is a statement *about an existing request*. A change arrives with no addressable target. And no component owns ordering, so requirement #2 has nowhere to live.

Sketch 2 — one runtime with subagents. The main agent answers easy things itself and spawns helpers for hard ones. Changes are natural here, because everything shares context. But the main runtime becomes a **serialization point**: every incoming change, every helper's result, and every delivery decision passes through one loop that is *also doing model inference*. Under a burst the backlog comes back — now bottlenecked on the one component you cannot replicate.

> THE ARCHITECTURAL CLAIM
>
> **Serialize the cheap thing. Parallelize the expensive thing.**

The brief's second sketch does the exact opposite: it serializes inference and parallelizes only the leaves. This is the central claim of the submission, so § 09 **measures** it rather than asserting it — there is a metric whose whole job is to falsify it.

The Sequencer is a pure function

> **Pure function** — code that takes input and returns output, touching nothing else: no files, no network, no clock. Given the same input it always returns the same thing.

The Sequencer is written as `handle(event) → [commands]`. It never calls a model, never reads the time (time arrives *on* each event), and never performs I/O. It returns a list of instructions — *start this worker*, *stop that one*, *ship this answer* — and something else carries them out.

This is not stylistic. It buys three concrete things:

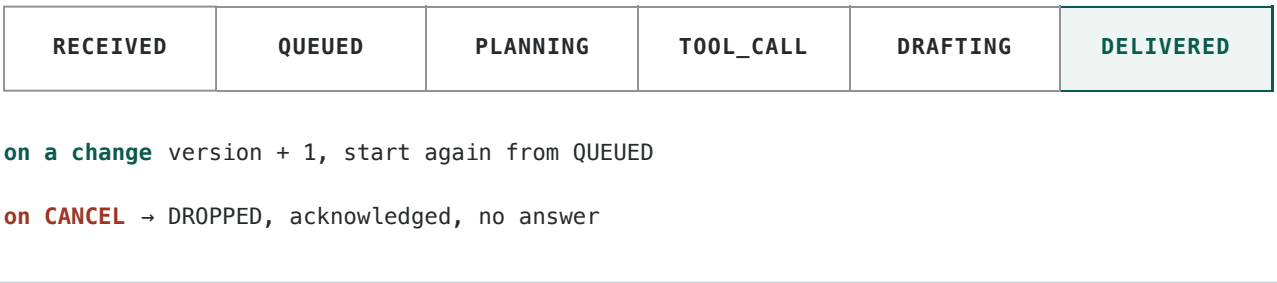
- **It makes the claim above structural.** "The Sequencer does no inference" stops being a promise and becomes something the shape of the function enforces.
- **The races in § 08 cannot happen.** One event is handled at a time, start to finish, so there is no moment where two things are half-applied.
- **It makes the testing in § 10 possible at all.** A pure function can be driven by a fuzzer thousands of times per second with no network calls.

The same Sequencer runs unchanged in the simulator and in live mode against real models. Only the thing driving it differs.

§ 06

How a request moves through the system

Every request carries a **version number** that goes up by one every time its specification changes. Every job started and every result returned is stamped with `(request, version)`. This one mechanism does most of the work in § 07.



QUEUED means waiting for a free worker slot. **PLANNING** is the model working out what it needs, **TOOL_CALL** is the expensive lookup, **DRAFTING** is writing the answer.

A queued request starts when **two** conditions hold: a worker slot is free, **and** everything it depends on has already been answered. The second condition is checked *before* a slot is taken, never while holding one — § 08 explains why that ordering is load-bearing.

§ 07

Handling changes mid-flight

Three mechanisms, and then the matrix that falls out of them.

(a) Changes apply at checkpoints, not instantly

> **Checkpoint** — a defined safe moment in a worker's execution. Here: just before each model call, and just after a tool call returns.

A worker has an inbox of pending changes and reads it only at checkpoints. So nothing is torn apart in the middle of an API call, and the system stays predictable enough to test.

Cost: a change may wait up to one tool call before it takes effect. That is stated as a tradeoff, not hidden. The alternative — interrupting a worker at an arbitrary instruction — buys a few seconds and costs the ability to reason about, or test, what state the worker was in when you stopped it.

(b) Version checking makes reliable cancellation unnecessary

The Sequencer ignores any result whose version is not the current one.

That sounds small and is the most important mechanism here. Reliably *stopping* work that is already running, across a network, is a genuinely hard problem — your stop signal races the work finishing, and sometimes loses. *Ignoring* work that came back out of date is trivial and cannot fail.

So we still try to stop superseded workers, to save money. But correctness never depends on that succeeding. This is not hypothetical: a stop signal sent while a worker is already writing its final answer *does* lose the race, the worker finishes, and it reports a result for a version that no longer exists. Version checking is what makes that harmless — and § 10 shows it failing on command when the check is turned off.

(c) Preserve work in proportion to what it cost

Tool calls are expensive in both time and tokens, and are frequently still valid after a change. A draft is cheap to rewrite. So a change that **preserves the premise** rewrites the answer over the tool results it already has, and one that **replaces the premise** throws them away and starts over.

The stage × relationship matrix

	QUEUED	PLANNING	TOOL CALL	DRAFTING	DELIVERED
AMEND `premise kept`	Rewrite in place; nothing lost	New version, start again	Let the tool call finish — already paid for — then apply at the next checkpoint	Rewrite the answer over the tool results we already have	Send an addendum
AMEND `premise replaced`	Replace in place	Start again	Start again, **throwing away the tool results**	Start again	New answer plus a visible "this replaces my earlier answer"
CANCEL	Drop, acknowledge	Stop, acknowledge	Let the tool call settle, then drop	Stop, acknowledge	Acknowledge *"already answered"* — no retraction
DUPLICATE	Attach the second asker; one answer, sent to both				Send the existing answer to the new asker

The Delivered column is the one most designs omit. A change can always arrive after the answer has already shipped. "Too late" is a real state with real required behaviour, not an error.

§ 08

Edge cases handled on purpose

These are the places a plausible-looking implementation quietly breaks. Each is handled explicitly and each has a test. **Four of the six were found by the fuzzer described in § 10**, not by being clever in advance.

THE TRAP	RESOLUTION
A cancelled parent with a waiting child	Ben asks "and the second biggest?", then cancels the first question. The follow-up is now waiting on an answer that will never come. When a request ends without ever answering, everything waiting on it is dropped with an explicit notice naming the question that died. **Silence is the one unacceptable outcome.**

THE TRAP	RESOLUTION
Waiting on the answer, not the request's status	A subtler version of the same thing. If a parent answers, is then amended (which makes it live again), and is then cancelled, its status is DROPPED — but the answer the child needs *already shipped*. Asking "is the parent currently delivered?" strands that child forever; asking "has the answer it needs been delivered?" does not. The gate is written the second way.
Dependency deadlock in the worker pool	If a waiting request could take a worker slot and *then* wait for its parent, a pool full of waiters would deadlock — every slot held by something that cannot proceed, and the thing it is waiting for unable to get a slot. The dependency check therefore happens at **admission**: a request never occupies a slot while waiting on another request.
The ready-but-undelivered window	Between "the worker finished" and "the ordering policy released the answer", a request has no worker but is not finished either. A change arriving in that window used to do two bad things at once: start a replacement worker while the old slot was already free (**breaking the pool limit**), and leave the now-obsolete answer sitting in the delivery buffer where a later release could still ship it. A change now always discards any undelivered answer, and always goes back through admission rather than starting a worker directly.
Two things happening at the same instant	A result arriving and a change arriving can land at the same moment. The Sequencer handles one event at a time, to completion, so there is no interleaving to get wrong — the concrete payoff of keeping the shared-state component free of model calls (§ 05).

THE TRAP	RESOLUTION
Dependency cycles from classifier error	A misclassification could make A wait for B while B waits for A — a loop nobody can ever get out of. Detected when the link is created, and broken by dropping the newer one. The result is a correct answer with weaker ordering, which is the right direction to fail.
Unbounded classifier input	Triage needs to see the live request set, and that set grows. It sees every unfinished request plus the last few answered ones. The live set is bounded; the chat history is not, and is not included.

Deadlock — a standstill where everyone is waiting for someone else and nothing can move. Classically: two people each holding one chopstick, each waiting for the other's.

§ 09

What we measure, and why

Everything below is recorded per scenario, per policy, and printed as a comparison table.

METRIC	WHAT IT TELLS YOU
Time to first / last answer	How long the burst feels. The user-perceived win.
Head-of-line delay	Total time that *finished* answers spent waiting on unfinished ones. Isolates the cost of ordering from the cost of the work.
Wasted tokens	Money spent on versions of requests that were never delivered. **The price of mutability, in dollars.**
Staleness	Answers shipped whose question had already changed. Must be **0** — a correctness check, not a performance number.
Misattribution exposure	Answers shipped with no "re: …" while more than one question was outstanding — the reader had to count positions to know what was being answered.

METRIC	WHAT IT TELLS YOU
Sequencer p99	How long the Sequencer spends per event. **Directly tests "serialize the cheap thing."** If this is not tiny, the central claim of § 05 is false.

p99 — the 99th percentile: the value that 99 out of 100 measurements come in under. Used instead of an average because averages hide the occasional very slow case, which is usually the one that matters.

The last three are what make this an engineering argument rather than a rhetorical one. Two of them are designed to be falsifiable: misattribution and Sequencer p99 would both go non-zero if the design were wrong about itself.

Ordering policies, compared head to head

Because § 03 made ordering a policy rather than a structure, these are interchangeable and comparable on identical inputs.

POLICY	BEHAVIOUR
SERIAL	One request at a time. **The honest baseline** — what the system does today.
STRICT_FIFO	Work in parallel, but hold each answer until everything that arrived before it is finished. Isolates the cost of literal ordering.
DEPENDENCY_ORDER	Ship as soon as ready, with attribution and acknowledgements; a request waits only for something it genuinely depends on. **Proposed default.**

SERIAL is in the list deliberately. Option 2 of the brief asks candidates to compare against "a simple baseline such as SPY alone"; the equivalent discipline here is to measure against the behaviour we claim to be improving on, not only against other clever designs.

§ 10

How we know it is correct

Measuring speed is the easy half. The harder question is whether the thing is *right*, and that needs a different kind of evidence.

Executable invariants

> **Invariant** — something that must be true of the system at all times, no matter what happens. "The pool never runs more workers than it has slots" is an invariant.

The properties this document claims are written as code that checks them, not as prose that asserts them.

INVARIANT	WHAT IT FORBIDS
`no_deadlock`	Any request still unfinished when everything has settled
`completeness`	A request the reader was acknowledged for that never produced any visible outcome
`no_stale`	Shipping an answer whose question had already changed
`non_contradiction`	The answer text changing without an addendum or replacement notice
`causal_consistency`	A dependent answer arriving before the answer it depends on
`attribution`	A policy that promises a requote shipping without one
`pool_bound`	More workers in flight than the pool allows
`tokens_accounted`	Shipping an answer that no worker ever actually computed

Randomised testing

> **Property-based testing** (or *fuzzing*) — instead of writing example cases by hand, you generate thousands of random inputs and assert that your invariants hold on every one. Hand-written tests cover the failures you thought of; this covers the ones you didn't.

Because the Sequencer is a pure function (§ 05) and the simulator is deterministic, a randomly generated chat can be replayed in well under a millisecond. So the test suite generates random message streams — arbitrary mixes of new questions, amendments, cancels, duplicates and references, at arbitrary times, under all three policies and every pool size — and asserts all eight invariants on each. A hundred thousand such chats run in a couple of minutes without a single network call.

This found four real bugs that the hand-written tests did not, all listed in § 08: the orphaned follow-up, the delivered-then-cancelled parent, the pool overrun in the ready window, and the stale answer surviving that

same window. It also falsified two of the invariants as too crude — which was worth as much, because it forced them to state the property precisely rather than approximately.

A correctness property you can break on command

> Claiming "staleness is always zero" is weak. Being able to make it non-zero on demand is strong. The stop-signal race in § 07b is modelled faithfully: a worker already writing its answer finishes regardless. Running the same scenario with version checking disabled ships the obsolete answer — and the invariant checker catches it, along with a second violation noticing that the delivered answer was never computed by any worker. Turn the check back on and both return to zero.

Scenario suite

SCENARIO	WHAT IT TARGETS
1	The § 01 transcript — everything at once; the headline demo
2	Burst of six simultaneous questions — parallelism and pool saturation
3	Slow Q1, fast Q2 — head-of-line blocking, in isolation
4	Amendment lands mid-tool-call — a change at the worst possible moment
5	Cancel after the draft is written — wasted-spend accounting
6	Amendment during the final draft — the stop-signal race; the version-check demo
7	Premise replaced mid-flight, then again after delivery — throwing work away, and the replacement notice

Tools are mocked with **controllable latency** — the simulator decides exactly when a tool call returns. This is a deliberate upgrade over real tools rather than a shortcut: reproducing "the amendment arrives *during* the tool call" requires controlling precisely that, and a real API makes the race unreproducible.

Workers also have a **live mode** that makes real `claude-opus-5` calls against a portfolio fixture, with real `claude-haiku-4-5` triage — so the demo shows the architecture doing the actual job, not just moving tokens around. Scenario replay uses mocks for determinism; live mode is for the demonstration.

§ 11

Assumptions

Stated as assumptions because several are load-bearing, and a reader should be able to reject one and immediately know what breaks.

ASSUMPTION	IF IT IS FALSE
Chats are independent	The Sequencer stops being per-chat, and the no-collisions property goes with it.
Within a chat, the only real ordering requirement is dependency	The thesis of § 03, and the most important assumption here. If a compliance rule demands literal arrival order, STRICT_FIFO already exists as a policy and we pay the blocking knowingly.
Tool calls cannot be interrupted once started	Conservative. If they can, cancellation gets cheaper and the Tool call column of § 07 simplifies.
Model calls can be interrupted	True over HTTP — though § 07b exists precisely because interruption can still lose the race.
The classifier is fallible	Its errors are made asymmetric by design (§ 04).

ASSUMPTION	IF IT IS FALSE
Answer volume per chat is low enough that acknowledgements are not spam	If false, acknowledgements collapse into a single edited status message.
Single process	The event log is the boundary where splitting across machines would go.

§ 12

What is built, and what is only designed

Five hours. The cut line is itself a design decision, so it is stated rather than hidden.

Built

- Request register, lifecycle, dependency links, and the Sequencer as a pure function
- Worker pool, change inboxes, checkpoint handling, version checking
- Triage over the five relationships of § 04, with the confidence asymmetry, and every decision auditable
- Delivery: attribution, acknowledgements, addenda, replacement notices
- Three policies, seven scenarios, the full metrics table
- Eight executable invariants, a randomised fuzzer, and the version-check demonstration
- Live mode: real Haiku triage and real Opus workers against a portfolio fixture

Designed, not built

- **MERGE / SPLIT relationships.** The highest implementation cost in the § 07 matrix — folding two running workers together, or spawning a second out of one — for the rarest real cases, and the two categories a small classifier is least reliable at telling apart. The matrix slot is designed; the code is not written. Adding them is additive, not structural.
- **BOUNDED_REORDER(k) policy.** Ship out of order, but never more than *k* places ahead of the oldest outstanding question. A real idea; three policies already prove the argument.
- **Fair sharing across chats.** One very busy chat can currently monopolise the worker pool. A per-chat cap is the obvious knob; real fair-share scheduling is not built.
- **Real Slack or Discord integration.** Authentication and webhooks are not what the brief grades. The replay harness demonstrates the same system better.
- **Durable storage beyond the event log.** Replay is what we need; surviving a crash is not under test.
- **Retries, backoff, rate limiting.** Real, orthogonal, and well understood.

§ 13

Known failure modes

Listed because a design document that claims no weaknesses is not describing a real system.

- **Genuinely ambiguous messages.** *"Actually, Q3"* may be an amendment or a new question; a human would ask which. The § 04 bias mitigates this; it does not solve it.
- **The ready-but-undelivered window is handled, not eliminated.** A change landing between "answer ready" and "answer shipped" now correctly discards the pending answer (§ 08), but the reader may still see an acknowledgement followed by a different answer than they were expecting. Correct, mildly disorienting.
- **Cross-chat pool starvation.** One very busy chat can monopolise the workers. Per-chat caps exist; fair sharing does not (§ 12).
- **Dependency cycles from classifier error.** Detected and broken (§ 08), at the cost of weaker ordering for that pair.
- **Unbounded chat history.** The live request set is bounded and the classifier's input is bounded by construction (§ 08), but a very long-lived chat will eventually need its transcript summarised.
- **The classifier is the weakest link.** Everything downstream depends on it relating messages to requests correctly, and that is a genuinely hard judgement. The asymmetric thresholds make its errors recoverable rather than destructive, and the decision log makes every call auditable — but neither makes it right.

Glossary

TERM	MEANING
Agent	A program that answers questions using a language model, calling tools to look things up along the way.
Tool call	The agent fetching external data — holdings, prices. The slow, expensive step.
Token	The unit language models are billed in, roughly a piece of a word. “Wasted tokens” are ones we paid for and threw away.
Message	One line somebody typed in the chat.
Request	One piece of work the agent owes an answer for. Several messages can relate to one request.
Parallel	Several requests being worked on at the same time.
Worker	One running job, answering one request.
Pool / slot	The bounded set of workers allowed to run at once; a request waits for a free slot.
FIFO	“First in, first out” — strict arrival order.
Head-of-line blocking	A slow item at the front of a queue delaying everything behind it.
Deadlock	A standstill where everything is waiting on something else and nothing can proceed.
Race	Two things happening at once where the outcome depends on which lands first.
Version	A counter on each request, incremented whenever its question changes. Used to recognise and ignore out-of-date results.
Stale	A result computed against a question that has since changed.

TERM	MEANING
Checkpoint	A defined safe moment in a worker's run where it reads pending changes.
Classifier	A model that sorts each message into one of the five relationships of § 04.
Confidence	The classifier's own estimate, 0 to 1, of whether it is right.
Pure function	Code that takes input and returns output without touching files, network, or the clock. Same input, same output, every time.
Deterministic	Produces exactly the same result every run — which is what makes replay and fuzzing possible.
Invariant	Something that must be true of the system at all times.
Property-based testing / fuzzing	Generating thousands of random inputs and asserting the invariants hold on all of them.
p99	The 99th percentile — the value 99 of 100 measurements fall under.
